09

SKT K-뉴딜 아카데미 · AI·자동화 기초

코드 리뷰 · 테스트 디버깅 · 회고

오전에 만든 파이프라인을 점검하고, 고치고, 마무리합니다.
1과목 AI·자동화 기초의 마지막 오후입니다.

오후에는 세 교시에 걸쳐 점검하고, 고치고, 마무리합니다

5 · 6교시는 LLM(엘엘엠) 을 부르지 않습니다.

리뷰 · 테스트 5교시 → 디버깅 6교시 → 완성 · 회고 7교시 → 확인 · ALEPH(알레프) 8교시

교시	무엇	LLM	만드는 파일
5교시	코드 리뷰와 단위 테스트	부르지 않음	`test_agent_core.py`
6교시	디버깅 — 에러가 있는 코드 고치기	부르지 않음	—
7교시	완성과 회고 · 깃허브에 올리기	부름 (7-2 · 7-3)	`docs/day08_retrospective.md`
8교시	오늘 만든 것 확인 · ALEPH 기본 과제	—	—

5교시에 사용하는 용어는 네 개입니다
같이 알아보면 좋은 용어: 회귀 오류

용어 (읽는 법)	뜻	5교시의 예
코드 리뷰 (code review)	정해 둔 체크리스트로 코드를 한 줄씩 읽는 것	비밀 · 설정 · 검증 · 실패 · 기록
단위 테스트 (unit test)	함수 하나가 맞게 도는지 자동으로 확인하는 것	`make_lines` 테스트
`assert` (어서트)	조건이 참이면 지나가고, 거짓이면 멈춤	`assert len("high") == 4`
`AssertionError`	`assert` 의 조건이 거짓일 때 나는 에러	`AssertionError: 메시지`

코드가 돌아간다고 맞는 것은 아니어서 리뷰와 테스트로 점검합니다

그래서 키 노출 · 실패 숨김처럼 돌아가도 틀린 코드를 찾아냅니다.

	코드 리뷰	단위 테스트
누가	사람	`assert` 를 적어 둔 코드
어떻게	체크리스트로 한 줄씩 읽음	「이 입력이면 이 값」 을 적어 둠
좋은 점	매번 같은 항목이라 빠뜨리지 않음	고친 뒤 한 번 돌리면 망가진 곳을 바로 앎

> 키가 코드에 직접 적혀 있거나 실패를 숨기는 코드도 잘 돕니다.

5교시에는 체크리스트로 리뷰하고 함수 셋을 테스트합니다

파이프라인이 쓰는 함수 셋이 맞게 도는지 LLM 없이 확인합니다.

① 리뷰 다섯 항목 → ② 첫 테스트 make_lines → **③ 판정 테스트 needs_approval** → ④ 실패 테스트 parse_llm_json

순서	하는 일	노트북	끝나면 보이는 값
① 리뷰	다섯 항목으로 통과 · 위반	문제 5-2	위반 3 개
② 첫 테스트	어제 만든 make_lines	문제 5-3	[테스트 통과] make_lines 2건
③ 판정 테스트	오전에 만든 needs_approval	문제 5-4	[테스트 통과] needs_approval 3건
④ 실패 테스트	10/6 에 만든 parse_llm_json	문제 5-5	[테스트 통과] parse_llm_json 3건

리뷰 체크리스트는
비밀 · 설정 · 검증 · 실패 · 기록 다섯 항목입니다

매번 같은 다섯 항목으로 보면 남의 코드에서도 빠뜨리지 않습니다.

항목	확인할 것	배운 날
비밀	API(Application Programming Interface) 키가 하드코딩 되어 있지 않은가	.env — 10/6
설정	모델 이름 · 주소가 config.json 에서 오는가	오늘 오전 2교시
검증	LLM 답을 그대로 믿지 않고 확인하는가	parse_llm_json — 10/6
실패	요청에 timeout 이 있는가	오늘 오전 3교시
기록	실패를 숨기지 않고 알리는가	[알림 실패] — 오늘 오전 3교시

1과목 규칙 다섯 가지에 이 코드를 맞춰 보면 세 곳이 규칙을 어겨 고쳐야 합니다

코드 리뷰 = 정해 둔 규칙에 코드를 한 줄씩 대 보는 것입니다.

규칙	지금 코드	판정	어떻게 고치나
비밀	`read_api_key()`	통과	—
설정	`"model": "gemini-3.5-flash-lite"`	위반	`"model": config["model"]`
검증	`parse_llm_json(…)`	통과	—
실패	`requests.post(URL, …)`	위반	`requests.post(…, timeout=10)`
기록	`except Exception: pass`	위반	`except Exception as e: print("[알림 실패]", type(e).__name__)`

`assert 조건` — 참이면 조용히 지나가고, 거짓이면 `AssertionError` 로 멈춥니다

거짓이면 멈추므로 틀린 값이 다음으로 넘어가지 않습니다.

```
assert 1 + 1 == 2          # 참 → 아무 일 없이 다음 줄로
print("여기까지 왔다")

assert 1 + 1 == 3          # 거짓 → 여기서 멈춤
print("이 줄은 실행되지 않는다")
```

```
여기까지 왔다
AssertionError
```

`1 + 1 == 2` 참	아무것도 출력하지 않고 다음 줄로 → 「여기까지 왔다」
`1 + 1 == 3` 거짓	`AssertionError` 를 내고 멈춤 → 아래 `print` 는 실행 안 됨

쉼표 뒤에 메시지를 붙이면 거짓일 때 **무엇이 틀렸는지** 함께 보여 줍니다

메시지가 있으면 여러 **assert** 가운데 어느 것이 틀렸는지 바로 압니다.

```
assert 1 + 1 == 3, "1 + 1 은 2 여야 한다"

AssertionError: 1 + 1 은 2 여야 한다
```

	`if 조건:`	`assert 조건, "메시지"`
하는 일	참 · 거짓에 따라 다른 길로 감	「꼭 참이어야 한다」를 확인
거짓이면	else 로 가거나 건너뜀 — 계속 실행	멈추고 메시지 출력
쓰는 곳	프로그램의 갈림길	함수가 맞게 만들어졌는지 시험 — 테스트

기대한 값을 코드에 적어 두면
파이썬이 대신 비교해 틀린 곳을 알려 줍니다

그래서 코드를 고친 뒤 다시 실행만 하면 망가진 곳을 바로 압니다.

```
def count_high(levels):           # 위험도 목록에서 high 가 몇 개인지 세는 함수
    n = 0
    for level in levels:
        if level == "high":       # 실수 - 대문자 "High" 는 못 셈
            n = n + 1
    return n

levels = ["High", "high", "low"]   # high 는 2개 - 나와야 할 값 2
print(count_high(levels))                      # 1 출력 - 맞는지는 사람이 판단
assert count_high(levels) == 2, "High 를 세지 못했다"
```

```
AssertionError: High 를 세지 못했다
```

try/except 는 에러를 받아 계속 가고, assert 는 에러 없이 틀린 값도 잡아 멈춥니다

테스트에서 찾는 것은 에러 없이 틀린 값이라서 assert 를 씁니다.

```
# 앞 장의 count_high — high 를 세는 함수인데 대문자 "High" 를 못 셈
count_high(["High", "high", "low"])   # 나와야 할 값 2 · 실제 1 · 에러는 안 남
```

	try / except — 실패해도 계속 가자	assert — 틀렸으면 지금 멈춰 알려 줘
언제 쓰나	프로그램이 돌아가는 중	내가 만든 함수를 시험할 때(테스트)
다루는 것	바깥 탓 실패 — 서버 꺼짐 · 파일 없음	내 코드의 실수 — 계산 · 조건이 틀림
에러가 나면	받아서 멈추지 않고 계속	—
에러 없이 틀린 값이면	못 잡음 — 그냥 지나감	잡아서 멈춤
오늘 쓴 곳	오전 notify_safe — 알림이 실패해도 보고서 저장	오후 테스트 — make_lines 등이 맞는지

조건이 거짓이면 AssertionError 로 멈춥니다

멈춘 줄의 메시지가 틀린 기대값을 알려 줍니다.

```
assert 1 + 1 == 2, "1 + 1 은 2 여야 한다"     # 참 - 다음 줄로
print("첫 번째 검사 통과")

assert 1 + 1 == 3, "1 + 1 은 2 여야 한다"     # 거짓 - 여기서 멈춘다
print("두 번째 검사 통과")                    # 실행되지 않는다

# 첫 번째 검사 통과
# AssertionError: 1 + 1 은 2 여야 한다
```

쓰는 것	결과
assert 조건, "메시지"	참이면 통과, 거짓이면 AssertionError: 메시지
assert 함수(입력) == 기대값	이것이 테스트 — 함수가 기대한 값을 리턴하는지 확인

make_lines 에 요약 한 건과 빈 리스트를 넣어 테스트합니다

보고서의 건별 줄을 만드는 함수라 \n 하나만 빠져도 보고서가 깨집니다.

```
sample = [{"id": "E01", "risk_level": "High", "summary": "로그인 실패 4회"}]

assert report_generator.make_lines(sample) == "- [HIGH] E01 로그인 실패 4회\n",
"make_lines 결과가 다르다"

assert report_generator.make_lines([]) == "", "빈 리스트는 빈 문자열이어야 한다"
print("[테스트 통과] make_lines 2건")
```

입력	기대값	확인하는 것
위험도가 High 인 한 건	"- [HIGH] E01 로그인 실패 4회\n"	대문자 · id · 요약 · \n 까지
빈 리스트 []	""	요약이 없으면 줄도 없음

`parse_llm_json` 은 깨진 답을 받아도 None 을 리턴해야 합니다

LLM 이 문장이나 빈 답을 보내도 멈추지 않고 None 을 리턴해야, 부르는 쪽이 `is None` 으로 실패를 알아챕니다.

```
fenced = "```json\n{\"tool\": \"lock_account\"}\n```"

assert llm_client.parse_llm_json(fenced) == {"tool": "lock_account"}, "코드 블록을
벗기지 못했다"
assert llm_client.parse_llm_json("그럴듯한 문장입니다") is None, "깨진 입력은 None
이어야 한다"
assert llm_client.parse_llm_json("") is None, "빈 답은 None 이어야 한다"
```

LLM 이 보낸 답	리턴해야 하는 것	왜
코드 블록에 싸인 JSON	`{"tool": "lock_account"}` 딕셔너리	```json 껍데기를 벗기고 읽어서 씀
JSON 이 아닌 문장	None	읽을 수 없음 → 「실패」 표시
빈 답 ""	None	받은 것이 없음 → 「실패」 표시

테스트를 test_agent_core.py 로 모으면 명령 한 번으로 확인합니다

그래서 코드를 고칠 때마다 망가진 곳이 없는지 바로 봅니다.

```
# test_agent_core.py — 5-3 · 5-4 · 5-5 의 assert 여덟 줄
import llm_client
import notifier
import report_generator
...
print("[테스트 통과] 8건 모두")
```

```
python test_agent_core.py
[테스트 통과] 8건 모두
```

⭐ 도전 5-6	미리 채운 데이터(sample · config · fenced)도 함께 옮김

5교시에 배운 것을 정리합니다

방법	하는 일	쓰는 것
코드 리뷰	매번 같은 항목으로 코드를 읽음	비밀 · 설정 · 검증 · 실패 · 기록
단위 테스트	입력과 기대값을 적어 두고 자동으로 확인	`assert 함수(입력) == 기대값, "메시지"`
테스트 파일	고칠 때마다 한 번에 확인	`python test_agent_core.py`

실습 ④ · 노트북 5교시 절 · 261008_pm_review_debug_retro.ipynb

문제 5-1 ~ 5-5 · 도전 ⭐5-6

① **맨 위 위치 확인 셀과 준비 셀 일곱을 먼저 실행합니다**
· 오전과 같은 가상환경을 커널로 고릅니다

② **문제 5-1 · 5-2 — assert 와 코드 리뷰**
· 5-1 은 결과를 먼저 적습니다

③ **문제 5-3 ~ 5-5 — 함수 셋 테스트**
· 5교시는 LLM 을 부르지 않습니다

6교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	6교시의 예
디버깅 (debugging)	에러의 원인을 찾아 고치는 일	마지막 줄부터 읽기
트레이스백 (traceback)	에러가 난 파일 · 줄 번호 · 에러 이름을 보여 주는 메시지	`KeyError: 'sevrity'`
문법 · 실행 에러 (syntax · runtime error)	실행 전에 막히거나, 실행하다 그 줄에서 멈춤	`SyntaxError` · `NameError`
논리 에러 (logic error)	멈추지 않고 틀린 결과를 냄	`"High" == "high"` → `False`

캡스톤에서는 정답이 없어서
에러 메시지에서 원인을 찾습니다

그래서 6교시 문제는 정답 대신 에러 메시지를 읽고 고치는 연습입니다.

	지금까지	캡스톤에서
에러가 나면	노트북 맨 아래 정답을 봄	정답이 없음
원인은	정답과 비교해 찾음	에러 메시지에 적혀 있음
6교시 문제	—	모두 에러가 있는 코드로 시작 — 먼저 그대로 실행

6교시에는 에러가 있는 코드 넷을 고칩니다

6교시에는 LLM 을 부르지 않습니다.

① 문법 에러 **SyntaxError(신택스 에러)** → ② 실행 에러 **KeyError(키 에러)** → ③ 실행 에러 **NameError(네임 에러)** → **④ 논리 에러 print 로 찾기**

순서	처음 실행하면	노트북	고치면 보이는 값
① 문법 에러	`SyntaxError: expected ':'`	**문제 6-2**	`# 야간 보안 관제 보고 (…)`
② 실행 에러	`KeyError: 'sevrity'`	**문제 6-3**	`[판단] brute_force → high`
③ 실행 에러	`NameError`	**문제 6-4**	`[완료] … 저장`
④ 논리 에러	`[요약] high 경보 0건`	**문제 6-5**	`[요약] high 경보 2건`

에러가 나면 마지막 줄부터 읽습니다

마지막 줄에 에러 이름과 내용이 있어서 원인을 가장 빨리 찾습니다.

순서	할 일
1	마지막 줄의 에러 이름과 내용을 읽음 — `KeyError: 'sevrity'`
2	그 위의 줄 번호로 가서 그 줄을 봄
3	에러가 없는데 결과가 이상하면 의심 가는 줄 위에 `print("[디버그]", 값)` 을 넣음
4	고친 뒤 다시 실행해 기대한 결과와 비교함

에러 없이 값이 나와도 틀린 결과라면 논리 에러입니다

에러가 없어 그대로 두면 틀린 보고서가 나갑니다.

```
levels = ["High", "high", "low"]
count = 0
for level in levels:
    if level == "high":
        count = count + 1
print(count)                    # 1
```

값	level == "high"	세었나
"High"	False	못 셈
"high"	True	셈
"low"	False	—

에러는 세 종류이고, 논리 에러가 가장 위험합니다

종류마다 찾는 법이 달라서 먼저 어느 에러인지 가립니다.

```
def make_title(today)          # 문법 에러 - 한 줄도 실행되지 않는다
print(judgment["sevrity"])     # 실행 에러 - 위 줄까지는 돌고 여기서 멈춘다
if level == "high":            # 논리 에러 - 에러 없이 틀린 결과가 나온다
```

종류	에러 메시지	찾는 법
문법 에러	SyntaxError — 실행 전에 나옴	^ 표시가 가리키는 자리를 봄
실행 에러	KeyError · NameError 등	마지막 줄의 에러 이름과 줄 번호를 봄
논리 에러	없음	print 로 실제 값을 출력해 기대와 비교

트레이스백은 아래에서 위로 읽습니다

마지막 줄의 에러 이름을 보면 무엇부터 확인할지 정해집니다.

```
Traceback (most recent call last):                ← 에러 보고의 시작
  File "pipeline.py", line 27, in run_pipeline    ← 어느 파일, 몇 번째 줄
    print(judgment["sevrity"])                    ← 그 줄의 코드
KeyError: 'sevrity'                               ← 여기부터 읽는다
```

마지막 줄	뜻	먼저 볼 곳
KeyError: 'sevrity'	딕셔너리에 그 키가 없음	키 이름의 철자
NameError: name 'x' is not defined	그 이름을 만든 적이 없음	만드는 줄이 위에 있는지
SyntaxError: expected ':'	문법이 틀림	^ 가 가리키는 자리

def 줄 끝에 : 이 빠지면 한 줄도 실행되지 않습니다

문법 에러는 셀 전체가 실행되지 않으므로 ^ 자리부터 고칩니다.

```
def make_title(today)
    return f"# 야간 보안 관제 보고 ({today})"

print(make_title("2026-10-08"))
# SyntaxError: expected ':'
```

	코드	결과
처음	def make_title(today)	SyntaxError: expected ':'
고친 뒤	def make_title(today):	# 야간 보안 관제 보고 (2026-10-08)

KeyError 가 나면 키 철자를 실제 키와 비교합니다

LLM 판단의 키 철자 하나만 틀려도 파이프라인이 그 자리에서 멈춥니다.

```
judgment = {"rule": "brute_force", "severity": "high", "tool": "lock_account"}
print(f"[판단] {judgment['rule']} → {judgment['sevrity']}")
# KeyError: 'sevrity'
```

	키	결과
딕셔너리에 있는 키	severity	—
코드에 쓴 키	sevrity — e 가 빠짐	KeyError: 'sevrity'
고친 뒤	severity	[판단] brute_force → high

NameError 는 만드는 줄을 쓰는 줄보다 위로 올려 고칩니다

줄 순서만 바꿔어도 멈추므로 만드는 줄이 위에 있는지 먼저 봅니다.

```
print(f"[완료] {filename} 저장")
filename = "daily_report_20261008.md"
# NameError: name 'filename' is not defined
```

	순서	결과
처음	print → filename = …	NameError: name 'filename' is not defined
고친 뒤	filename = … → print	[완료] daily_report_20261008.md 저장

> 글자는 하나도 틀리지 않았습니다. 파이썬은 위에서 아래로 한 줄씩 실행합니다.

논리 에러는 print 로 실제 값을 출력해 찾습니다

에러 메시지가 없어서 실제 값을 출력해 봐야 기대와 다른 곳이 보입니다.

```
level = "High"
print("[디버그]", level, level == "high", level.lower() == "high")
# [디버그] High False True
```

할 일	쓰는 것
의심 가는 줄 바로 위에 값을 출력함	print("[디버그]", 변수)
기대한 값과 비교함	"high" 를 기대했는데 "High" 가 출력됨
고친 뒤에는 디버그 줄을 지움	보고서에 섞이지 않게

[디버그] 로 출력하면 원인이 보이고, .lower() 로 고칩니다

대소문자가 다르면 high 경보를 0건으로 셉니다.

```
high_count = 0
for j in judgments:
    print("[디버그]", j["severity"])
    if j["severity"].lower() == "high":
        high_count = high_count + 1
print(f"[요약] high 경보 {high_count}건")
```

	출력
처음 (디버그 줄만 넣음)	[디버그] medium · [디버그] High · [디버그] HIGH · high 경보 0건
.lower() 를 넣은 뒤	[요약] high 경보 2건

도전 6-6 — 에러 없이 틀린 세 곳을 찾습니다

그대로 두면 경고가 빠지고 순서가 뒤집히고 줄이 붙은 보고서가 나갑니다.

규칙	틀린 코드	고친 코드	틀리면 보이는 것
high 가 2건 이상이면 경고	`high_count > 2`	`high_count >= 2`	경고 줄이 안 나옴
위험한 것부터	`["low", "medium", "high"]`	`["high", "medium", "low"]`	**low 가 맨 위**
한 건에 한 줄	`f"- [{level}] …"`	`f"- [{level}] …\n"`	네 건이 한 줄로 붙음

> 먼저 그대로 실행해 기대 결과와 다른 곳을 셋 찾습니다. 실행은 멈추지 않습니다.

6교시에 배운 것을 정리합니다

논리 에러는 **print** 로 찾고, **assert** 로 다시 생기지 않게 합니다.

종류	보이는 것	오늘 고친 것
문법 에러	SyntaxError — 한 줄도 안 돈다	def 끝의 :
실행 에러	KeyError · NameError — 그 줄에서 멈춘다	키 철자 · 줄 순서
논리 에러	에러 없이 틀린 값	.lower() · >= · 정렬 순서 · \n

실습 ⑤ · 노트북 6교시 절 · 261008_pm_review_debug_retro.ipynb

문제 6-1 ~ 6-5 · 도전 ⭐6-6

① **문제 6-1 — 결과를 먼저 적습니다**
· 에러 없이 틀리는 코드

② **문제 6-2 ~ 6-4 — 먼저 그대로 실행해 에러를 읽습니다**
· 마지막 줄부터

③ **문제 6-5 — [디버그] 로 출력해 원인 찾기**
· 6교시는 LLM 을 부르지 않습니다

7교시에 사용하는 용어는 네 개입니다

용어 (읽는 법)	뜻	7교시의 예
회고 (retrospective)	한 일을 돌아보고 다음에 볼 기록을 남기는 것	`docs/day08_retrospective.md`
장애 허용 (fault tolerance)	한 부분이 실패해도 전체가 멈추지 않게 하는 것	알림이 실패해도 보고서는 남음
`../` (닷 닷 슬래시)	한 칸 위 폴더	`../docs/`
`os.path.exists`	그 파일이 있으면 `True`	`os.path.exists(파일)`

1과목의 마지막 시간 —
다시 돌려 보고, 기록하고, 올립니다

그래서 캡스톤에서 막힐 때 다시 볼 코드와 기록이 남습니다.

오늘 확인할 것	문제
명령 한 번으로 보고서 → 알림까지 간다	7-2
알림 서버가 꺼져도 보고서는 남는다	7-3
설정만 바꾸면 코드를 고치지 않아도 동작이 바뀐다	⭐7-6
8일 동안 무엇이 어려웠는지 기록한다 — 오늘이 아니면 잊음	7-4
오늘의 산출물을 깃허브에 올린다	7-5

7교시에는 두 경우로 돌리고 회고를 써서 올립니다

LLM 을 부르는 문제는 7-2 · 7-3 입니다. 둘 다 한 번 실행에 두 번 부릅니다.

① 정상 **config.json** → ② 알림 장애 **config_off.json** → ③ 회고 **day08_retrospective.md** → ④ 올리기 **깃허브**

순서	하는 일	노트북	끝나면 보이는 값
① 정상	처음부터 끝까지 + 소제목 확인	문제 7-2	`##` 한눈에 보기 등 소제목 셋
② 알림 장애	꺼진 서버 주소로 돌림	문제 7-3	보고서가 남았나: True
③ 회고	1과목 회고를 내 말로	문제 7-4	Writing ../docs/…
④ 올리기	명령어 또는 웹 화면	문제 7-5	내 저장소에 파일이 보임

POST 는 데이터를 서버에 보내 처리를 요청합니다

① 내 프로그램
POST + 데이터

→

② 서버
데이터로 처리

→

③ 응답
처리 결과

POST 예	보내는 데이터	서버가 하는 일
로그인	아이디 · 비밀번호	맞는지 확인
새 회원 · 게시물	이름 · 글 내용	새 자료를 만듦
파일 업로드	파일	받아서 저장
결제	카드 · 금액	결제를 실행
LLM 에게 질문	질문 글	답을 만들어 리턴
분석 · 계산	숫자 · 로그	계산을 실행
오늘 `notify()`	`{"text": 메시지}`	[알림 받음] 출력

알림 서버는 요청을 기다리는 프로그램이라, 켜 둔 채 다른 곳에서 파이프라인을 돌립니다

서버가 켜져 있는 동안 그 터미널은 명령을 받지 않습니다. 그래서 실행은 다른 곳에서 합니다.

① 터미널 A
`python alert_server.py`

→

② 기다리는 중
프롬프트가 안 돌아옴 = 정상

→

③ 노트북 · 터미널 B
`run_pipeline` → POST

→

④ 터미널 A
`[알림 받음]` 출력

> 터미널 A 를 닫거나 `Ctrl + C` 를 누르면 서버가 꺼져 알림이 실패합니다. 이 경우를 문제 7-3 에서 일부러 해 봅니다.

알림 서버를 켜고 시작하되, 이미 켜져 있으면 그대로 둡니다

알림 서버가 켜져 있어야 7-2 에서 알림 도착까지 확인할 수 있습니다.

```
python alert_server.py
 * Running on http://127.0.0.1:5001
```

보이는 것	뜻	할 일
Running on http://127.0.0.1:5001	켜짐	그 터미널은 그대로 둠
Address already in use	오전에 켠 서버가 아직 떠 있음	켜진 그대로 씀 — 끄지 않음

> agent_core 폴더를 오른쪽 클릭 › **Open in Integrated Terminal** 로 엽니다. 오전 3.1 과 같습니다.

설정에 빠진 키가 있으면
파이프라인은 더 가지 않습니다

빠진 키를 먼저 잡으면 LLM 을 부르기 전에 멈춰 호출을 아낍니다.

```
import notifier
config = notifier.load_config("config_broken.json")
print(config is None)
# [설정 오류] 빠진 키: report_folder
# True
```

단계	보이는 것
load_config	[설정 오류] 빠진 키: report_folder → None
run_pipeline 이라면	[중단] config.json 을 고친 뒤 다시 실행하세요 — **LLM 을 부르지 않음**

pipeline.run_pipeline 으로 끝까지 돌리고 소제목 세 줄을 확인합니다

명령 한 번으로 보고서와 알림까지 가는지 봅니다.

```
pipeline.run_pipeline("config.json", "2026-10-08")    # LLM 두 번

with open("daily_report_20261008.md", encoding="utf-8") as f:
    text = f.read()
for line in text.split("\n"):
    if line.startswith("## "):
        print(line)
```

어디	보이는 것
노트북	[보고서] daily_report_20261008.md 저장 · 사람 확인 필요 1건
소제목	## 한눈에 보기 · ## 총평 · ## 건별 내역 (위험한 것부터)
알림 서버 터미널	[알림 받음] [보고서] …

알림 서버가 꺼져 있어도 보고서는 끝까지 저장되는지 확인합니다

알림이 실패해도 보고서는 잃지 않는지 확인합니다.

```
config["webhook_url"] = "http://127.0.0.1:5999/alert"    # 아무 서버도 없는 주소
with open("config_off.json", "w", encoding="utf-8") as f:
    json.dump(config, f, ensure_ascii=False, indent=2)

pipeline.run_pipeline("config_off.json", "2026-10-09")
print("보고서가 남았나:", os.path.exists("daily_report_20261009.md"))
```

차례	보이는 것
알림	[알림 실패] ConnectionError — 오전 3교시의 notify
보고서	[보고서] daily_report_20261009.md 저장 …
확인	보고서가 남았나: True

코드는 그대로, 설정 파일만 바꿔 동작을 바꿉니다

그래서 캡스톤에서 고객사가 바뀌어도 `config.json` 만 고치면 됩니다.

설정 파일	바꾼 값	결과
`config.json`	—	사람 확인 필요 1건 · **알림 도착**
`config_off.json`	`webhook_url` → **꺼진 서버**	**알림 실패** · 보고서는 남음
`config_low.json` **(⭐7-6)**	`approve_severity` → `low`	사람 확인 필요 6건 — **모든 경보를 사람이 봄**

> **캡스톤에서 고객사가 바뀌면 이 파일만 바꿉니다.**

회고는 한 칸 위 docs 폴더에 저장합니다

날짜 기록이 있는 docs 에 모아 한곳에서 찾습니다.

```
os.makedirs("../docs", exist_ok=True)
print("회고를 저장할 폴더:", os.path.abspath("../docs"))

%%writefile ../docs/day08_retrospective.md
# 1과목 회고 (이름)
```

경로	저장되는 곳
day08_retrospective.md	지금 폴더(agent_core) 안
../docs/day08_retrospective.md	한 칸 위(security-agent-toolkit)의 docs 안

⚠ docs 폴더가 없으면 저장되지 않습니다. 바로 위 셀이 폴더를 먼저 만듭니다.

1과목 회고는 네 칸을 채웁니다

8일 동안 막힌 것을 남겨 두면 캡스톤에서 막힐 때 다시 열어 봅니다.

칸	쓸 것
내가 만든 것	오늘까지 만든 파이프라인이 무엇을 하는지 한두 줄
가장 어려웠던 것과 해결법(중요)	8일 가운데 가장 오래 막혔던 것
리뷰에서 찾은 것과 고친 것	5교시 5-2 · 6교시에서 찾은 것
캡스톤에서 더 해 보고 싶은 것	다음에 붙여 보고 싶은 기능

> 제목의 (이름) 을 내 이름으로 바꿉니다.

산출물은 깃허브에 올립니다

깃허브에 올려 두면 캡스톤에서 오늘 만든 파일을 다시 꺼내 씁니다.

	명령어로 (아침 과제 7 을 한 사람)	웹 화면으로 (아직인 사람)
어디서	security-agent-toolkit 에서 연 터미널	브라우저의 내 저장소
순서	status → add → commit → push	폴더 안으로 들어가 Add file › Upload files
주의	git status 에 .env 가 보이면 멈춤	agent_core 와 docs 를 따로 올림

> **올릴 것:** config.json · notifier.py · pipeline.py · test_agent_core.py · daily_report_20261008.md · docs/day08_retrospective.md

명령어로 올릴 때는 `cd ..` 로 한 칸 올라가 네 명령을 입력합니다

agent_core 와 docs 를 한 번에 올리려면 둘을 담은 폴더에서 입력합니다.

```
cd ..
git status
git add agent_core docs
git commit -m "Add day 8 pipeline, tests and retrospective" # -m(message) 뒤가 커밋 메시지
git push
```

`cd ..`	agent_core 에서 연 터미널이면 먼저 한 칸 위로
`git status`	.env 가 보이면 add 하지 않고 멈춤 — 아침 과제 7 의 6번
확인	브라우저의 내 저장소에서 agent_core 와 docs 안에 파일이 보임

7교시에 배운 것을 정리합니다

확인한 것	어떻게
명령 한 번으로 끝까지	`pipeline.run_pipeline("config.json", 날짜)`
알림이 실패해도 보고서는 남는다	config_off.json — 주소만 바꿨다
설정이 동작을 바꾼다	config_low.json — 기준만 바꿨다
기록	docs/day08_retrospective.md

실습 ⑥ · 노트북 7교시 절 · 261008_pm_review_debug_retro.ipynb

문제 7-1 ~ 7-5 · 도전 ⭐7-6

① **알림 서버를 켜고 문제 7-1 ~ 7-3**
· 7-2 · 7-3 은 LLM 을 두 번씩 부릅니다

② **문제 7-4 — 1과목 회고**
· 바로 위 폴더 셀을 먼저 실행합니다

③ **문제 7-5 — 깃허브에 올리기**
· 명령어 또는 웹 화면

8교시 — 1과목의 산출물 아홉 개가 있는지 확인합니다

새로 배우는 것은 없습니다. 없음 이 나오면 그 파일을 만드는 문제로 돌아갑니다.

> **노트북 8교시의 확인 셀을 실행합니다.** 있음 : **이 아홉 줄 나오면 됩니다.**

파일	만든 날
`llm_client.py`	**10/6(화)**
`event_summarizer.py` · `report_generator.py`	**10/7(수)**
`config.json` · `notifier.py` · `pipeline.py`	오늘 오전
`test_agent_core.py` · `daily_report_20261008.md`	오늘 오후
`../docs/day08_retrospective.md`	오늘 7교시 — docs 폴더

1과목에서 만든 agent_core 는 캡스톤에서 그대로 다시 씁니다

그래서 2과목에서도 agent_core 는 지우지 않고 그대로 둡니다.

파일	하는 일	만든 날
llm_client.py	LLM 을 부르고 답을 JSON 으로 읽음	10/6
event_summarizer.py	경보를 묶음으로 요약하고 위험도순으로 정렬	10/7
report_generator.py	총평을 받고 보고서를 만듦	10/7
config.json	모델 · 정책 · 폴더 · 알림 주소	10/8
notifier.py	설정을 검사하며 읽고 알림. 실패해도 멈추지 않음	10/8
pipeline.py	위 모두를 명령 한 번으로 이음	10/8
test_agent_core.py	고칠 때마다 망가진 곳이 없는지 봄	10/8

관심 있는 주제를 계속 고민해 두었다가 내년 「모두의 창업」에 지원해 봅니다

지금 시간을 쓸 필요는 없습니다.

주제만 마음에 두고, 2027년 모집 때 지원하면 좋은 경험 · 스펙이 됩니다.

무엇	중소벤처기업부의 창업 지원사업 — 아이디어 하나로 지원
누가	사업자등록이 없는 예비창업자 · 직장인도 가능 · 팀으로도 가능 (2026년 2차 공고 기준)
받는 것	1라운드 활동자금 200만 원 + AI 도구 · 멘토링 → 단계를 통과하면 시제품 비용 최대 2천만 원
2027년	선발 1만 5천 → 2만 명, 1라운드 200만 → 500만 원으로 확대 추진 — 정부 예산안, 12월 국회에서 확정

🔗 modoo.or.kr — 공고 · 일정은 사이트에서 확인합니다. 내년부터 예비창업패키지도 이 사업으로 합쳐질 예정입니다.

실습 ⑦ · 8교시 남은 시간

ALEPH 기본 과제 12개 풀기

각 과제에 투입할 수 있는 시간을 정하고, 그 안에 끝나도록 AI 를 활용합니다.
앞 과제가 승인돼야 다음 과제가 열립니다. 자기 차례의 과제부터 합니다.

과제	제목	계획 시간	과제	제목	계획 시간
T01	나를 소개하는 한 페이지	4~5h	T07	플랜두씨 다이어리 2 — 인증	6~8h
T02	내가 설계한 미니게임	6~7h	T08	내 소개 페이지에 패스키 달기	6~7h
T03	짤·카드 스튜디오	4~5h	T09	자동화 관제소	5~6h
T04	오늘의 진짜 정보판	6~7h	T10	AI와 함께 쓰는 첫 논문	8~10h
T05	대화가 끊겨도 이어지는 프로젝트	4~5h	T11	3만자 장편소설	6~8h
T06	플랜두씨 다이어리 1	8~10h	BR-A / B	마지막 과제 — 둘 중 하나	4~7h

다음 주 10/12(월) 부터
2과목 네트워크 · 제로 트러스트 운영을 배웁니다

새 과목은 새 폴더에서 하므로 오늘 완성한 `agent_core` 는 그대로 남습니다.

	1과목 (9/22 ~ 오늘)	2과목 (10/12 ~)
과목	AI·자동화 기초	네트워크 · 제로 트러스트 운영
폴더	`agent_core`	`network_zt` — 새로 만듦
agent_core	오늘 완성	그대로 두고 캡스톤에서 다시 씀

🔔 **17:30 — 수업을 마치고 오후 리추얼로 넘어갑니다. 리추얼은 20분이고 17:50 에 끝납니다.**